جزوه درجه یک سیستم عامل

فصل ۳ - بخش ۷: مثال‌هایی از سیستم‌های IPC (بخش الف)

در بخش‌های ۴ تا ۶، ما دو مدل اصلی ارتباط بین پردازه‌ای (IPC) را به صورت تئوری یاد گرفتیم: ۱. **حافظه مشترک (تخته سیاه):** سریع، اما مدیریتش سخت. ۲. **عبور پیغام (اداره پست):** کندتر، اما امن و ساده.

حالا در این بخش می‌خواهیم ببینیم این دو مدل در سیستم‌عامل‌های واقعی (مثل لینوکس، macOS و ویندوز) چطور پیاده‌سازی شده‌اند.

۳-۷-۱: حافظه مشترک پوزیکس (POSIX)

"پوزیکس" (POSIX) استانداردی است که باعث می‌شود کدهای نوشته شده برای یک سیستم‌عامل شبه-یونیکس (مثل لینوکس) روی سیستم‌عامل دیگر (مثل macOS) هم اجرا شود.

استاندارد پوزیکس برای پیاده‌سازی "حافظه مشترک" (مدل تخته سیاه) از مفهومی به نام **"فایل‌های نگاشته به حافظه" (Memory-mapped Files)** استفاده می‌کند.

ایده کلیدی: به جای اینکه مستقیماً یک تیکه از RAM را به اشتراک بگذاریم، سیستم عامل یک "فایل مجازی" روی دیسک (که البته واقعاً روی RAM است) ایجاد می‌کند. سپس هر پردازه که بخواهد از آن حافظه مشترک استفاده کند، آن فایل را در فضای آدرس خودش "نگاشت" (Map) می‌کند. انگار که هر دو پردازه، یک فایل یکسان را باز کرده‌اند و می‌توانند همزمان در آن بنویسند و بخوانند.

مراحل پیاده‌سازی (مثال تولیدکننده):

1. **`shm_open()` (ایجاد اتاق):**
   - این تابع یک "شِئ حافظه مشترک" (یک فایل مجازی) با یک اسم مشخص (مثلاً "OS") ایجاد می‌کند و یک "توصیف‌گر فایل" (File Descriptor یا `fd`) به ما برمی‌گرداند.
   - `O_CREAT`: یعنی اگر وجود نداشت، آن را بساز.
   - `O_RDWR`: یعنی برای خواندن و نوشتن باز شود.
1. **`ftruncate()` (تعیین اندازه اتاق):**
   - بعد از ساختن فایل، باید بگوییم چقدر بزرگ باشد (مثلاً ۴۰۹۶ بایت).
1. **`mmap()` (گرفتن کلید اتاق):**
   - این مهم‌ترین تابع است.
   - این تابع آن فایل (`fd`) را به فضای حافظه پردازه ما "نگاشت" می‌دهد و یک **اشاره‌گر (Pointer)** به ما برمی‌گرداند.
   - از این به بعد، ما با آن فایل کاری نداریم. هر چیزی که در آن **اشاره‌گر** بنویسیم، مستقیماً در حافظه مشترک نوشته می‌شود و برای بقیه پردازه‌ها قابل دیدن است.
   - `PROT_WRITE`: یعنی می‌خواهیم در آن بنویسیم.
   - `MAP_SHARED`: یعنی تغییراتی که ما می‌دهیم، برای بقیه هم قابل رؤیت باشد.
1. **`sprintf(ptr, "...")` (نوشتن روی تخته سیاه):**

- حالا به سادگی با استفاده از آن اشاره‌گر (`ptr`)، داده‌ها را در حافظه مشترک می‌نویسیم (مثلاً "Hello World!").

و مصرف‌کننده چکار می‌کند؟ مصرف‌کننده دقیقاً همین مراحل را طی می‌کند، با این تفاوت که:

- در `shm_open` از `O_CREAT` استفاده نمی‌کند (چون اتاق قبلاً ساخته شده) و آن را فقط برای خواندن (`O_RDONLY`) باز می‌کند.
- در `mmap` از `PROT_READ` (فقط خواندن) استفاده می‌کند.
- به جای `sprintf` (نوشتن)، از `printf` (خواندن) از آن اشاره‌گر استفاده می‌کند.
- در نهایت، `shm_unlink()` را صدا می‌زند تا آن "اتاق" (شئ حافظه مشترک) را از سیستم حذف کند.

۳-۷-۲: عبور پیغام در سیستم‌عامل Mach

"Mach" یک هسته (Kernel) است که پایه‌ی سیستم‌عامل‌های مدرن اپل (macOS و iOS) را تشکیل می‌دهد.

مدل ارتباطی در Mach **فقط و فقط** بر اساس **"عبور پیغام" (مدل اداره پست)** است. حتی کارهای داخلی خودِ سیستم‌عامل هم با پیغام انجام می‌شود.

مفاهیم کلیدی Mach:

- **پورت (Port) = صندوق پستی (Mailbox):**
  - پیغام‌ها به "پورت"ها ارسال می‌شوند.
  - پورت‌ها **یک‌طرفه** هستند. (برای ارتباط دوطرفه، باید دو پورت داشته باشیم؛ یکی برای رفت، یکی برای برگشت).
  - **مهم:** هر پورت می‌تواند **چندین فرستنده** داشته باشد، اما **فقط و فقط یک گیرنده** دارد.
- **مالکیت و حقوق پورت (Port Rights):**
  - پردازه‌ای که یک پورت را ایجاد می‌کند، **"مالک" (Owner)** آن پورت می‌شود.
  - **فقط مالک** پورت حق دریافت (`MACH_PORT_RIGHT_RECEIVE`) پیغام از آن پورت را دارد.
  - مالک می‌تواند به پردازه‌های دیگر، حق ارسال (`MACH_PORT_RIGHT_SEND`) به آن پورت را بدهد.
- **پورت‌های ویژه (از بدو تولد):**
  - هر پردازه (وظیفه یا Task) وقتی ایجاد می‌شود، دو پورت ویژه می‌گیرد:
    1. **`Task Self Port` (پورت خودی):** برای اینکه پردازه بتواند به "هسته" (Kernel) پیغام بفرستد.
    2. **`Notification Port` (پورت اطلاع‌رسانی):** برای اینکه "هسته" بتواند رخدادها را به پردازه اطلاع دهد.
- **پیدا کردن همدیگر (Bootstrap Server):**
  - یک پردازه چطور پورت یک پردازه دیگر را پیدا کند؟
  - یک "سرور ثبت" به نام `Bootstrap Server` وجود دارد. پردازه‌ها پورت‌های عمومی خود را در آنجا "ثبت" می‌کنند تا بقیه بتوانند آن‌ها را پیدا کنند.
- **ساختار پیغام (نامه):**
  1. **سرآیند (Header):** اطلاعات مدیریتی (اندازه پیغام، پورت مقصد، و پورت مبدأ که همان "آدرس برگشتی" برای جواب است).
  2. **بدنه (Body):** داده‌های واقعی.

- **بهینه‌سازی جادویی Mach (جلوگیری از کپی):**
  - گفتیم عبور پیغام "کند" است چون باید داده‌ها از فرستنده به هسته "کپی" شوند، و بعد از هسته به گیرنده "کپی" شوند (Double Copy).
  - **راه حل Mach:** اگر فرستنده و گیرنده روی یک ماشین باشند، Mach از یک ترفند "مدیریت حافظه مجازی" استفاده می‌کند.
  - به جای کپی کردن داده‌ها، Mach فقط آن بخش از حافظه فرستنده را در فضای آدرس گیرنده **"نگاشت" (Map)** می‌دهد.
  - در عمل، هیچ داده‌ای کپی نمی‌شود! هر دو به یک مکان مشترک در حافظه نگاه می‌کنند. این کار باعث می‌شود عبور پیغام در Mach به طرز شگفت‌انگیزی **سریع** و در حد "حافظه مشترک" باشد.

۳-۷-۳: ویندوز و ALPC

ویندوز هم از یک مدل "عبور پیغام" پیشرفته، بهینه‌سازی شده و بسیار سریع به نام **ALPC (Advanced Local Procedure Call)** استفاده می‌کند. (یعنی "فراخوانی روال محلی پیشرفته").

ALPC برای ارتباط بین پردازه‌ها *روی همان ماشین* طراحی شده است.

نحوه کار (مدل کارفرما - کارگزار):

ویندوز هم مثل Mach از "پورت" استفاده می‌کند، اما با دو نوع پورت:

1. **پورت اتصال (Connection Port):**
   - این یک پورت "عمومی" و "شناخته‌شده" است که کارگزار (سرور) آن را منتشر می‌کند (مثل شماره تلفن گویای یک شرکت).
   - کارفرما (کلاینت) که می‌خواهد به سرویس وصل شود، ابتدا به این پورت عمومی یک "درخواست اتصال" می‌فرستد.
1. **پورت ارتباط (Communication Port):**
   - وقتی کارگزار درخواست اتصال را دریافت کرد، یک "کانال خصوصی" برای آن کارفرما ایجاد می‌کند.
   - این کانال خصوصی شامل **یک جفت** پورت ارتباطی است (یکی برای رفت، یکی برای برگشت).
   - کارگزار این پورت‌های خصوصی را به کارفرما می‌دهد.
   - از این به بعد، تمام ارتباطات کارفرما و کارگزار از طریق این کانال خصوصی و امن انجام می‌شود و کاری به پورت اتصال عمومی ندارند.